



ESCUELA TÉCNICA SUPERIOR DE
INGENIERÍA INFORMÁTICA

ESCUELA TÉCNICA SUPERIOR DE
INGENIERÍA DE TELECOMUNICACIÓN

Fundamentos de Inteligencia Artificial

Búsquedas No informadas

José A. Montenegro

5 de febrero de 2026



Contenido

Búsqueda de Soluciones Informadas

Algoritmo voraz

Búsqueda A*

Puzzle 8



Búsqueda de Soluciones Informadas





Búsquedas informadas

- Estrategias de búsqueda informada puede encontrar soluciones de manera más eficiente que una estrategia no informada.
- Selecciona un nodo para la expansión basada en una función de evaluación $f(n)$.
 - Normalmente se expande el nodo con la evaluación más baja.
- **Función heurística $h(n)$** , forma más común de transmitir el conocimiento adicional del problema al algoritmo de búsqueda.
 - coste estimado del camino más barato desde el nodo n a un nodo objetivo.



Algoritmo

```
function BEST-FIRST-SEARCH(problem, f) returns a solution node or failure  
  node  $\leftarrow$  NODE(STATE=problem.INITIAL)  
  frontier  $\leftarrow$  a priority queue ordered by f, with node as an element  
  reached  $\leftarrow$  a lookup table, with one entry with key problem.INITIAL and value node  
  while not IS-EMPTY(frontier) do  
    node  $\leftarrow$  POP(frontier)  
    if problem.IS-GOAL(node.STATE) then return node  
    for each child in EXPAND(problem, node) do  
      s  $\leftarrow$  child.STATE  
      if s is not in reached or child.PATH-COST < reached[s].PATH-COST then  
        reached[s]  $\leftarrow$  child  
        add child to frontier  
  return failure
```

```
function EXPAND(problem, node) yields nodes  
  s  $\leftarrow$  node.STATE  
  for each action in problem.ACTIONS(s) do  
    s'  $\leftarrow$  problem.RESULT(s, action)  
    cost  $\leftarrow$  node.PATH-COST + problem.ACTION-COST(s, action, s')  
    yield NODE(STATE=s', PARENT=node, ACTION=action, PATH-COST=cost)
```

Este es un algoritmo genérico en el cual elegimos un nodo, n , con el menor valor de una función de evaluación, $f(n)$. En los algoritmos de búsqueda se utilizan tres tipos de colas:

- **Cola de prioridad** - Una cola de prioridad selecciona primero el nodo con el coste mínimo según una función de evaluación, $f(n)$.
- **Cola FIFO** - Una cola FIFO o cola del primero que entra es el primero que sale. Corresponde con la implementación **búsqueda amplitud**.
- **Cola LIFO** - Una cola LIFO o cola de último en entrar primero en salir (también conocida como pila) saca primero el nodo añadido más recientemente. Corresponde con la implementación **búsqueda profundidad**.



Ejemplo Mapa Rumanía

Estados : Ciudades mapa

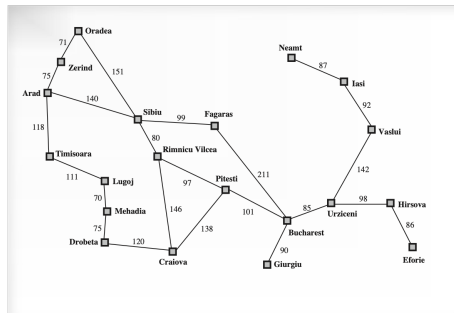
Estado Inicial : In(Arad)

Función Sucesor : $\text{Result}(\text{In}(\text{Arad}), \text{Go}(\text{Zerind})) = \text{In}(\text{Zerind})$

Acciones : $\{\text{Go}(\text{Sibiu}), \text{Go}(\text{Timisoara}), \text{Go}(\text{Zerind})\}$

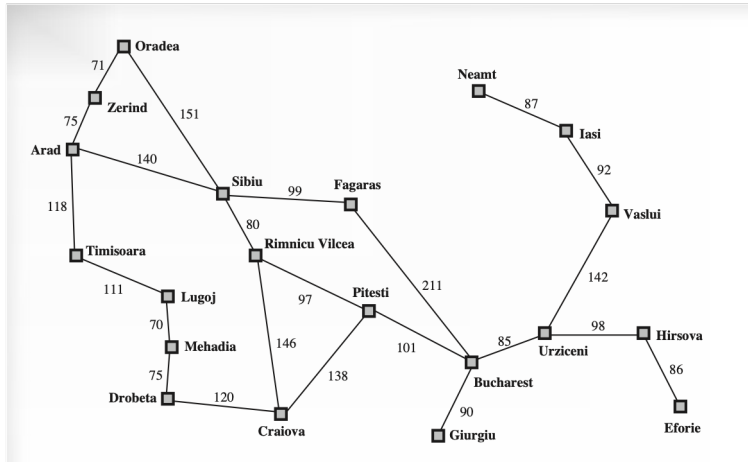
Objetivo : In(Bucharest)

Coste Camin : ver mapa





Ejemplo Mapa Rumanía





Algoritmo voraz





Algoritmo voraz (greedy)

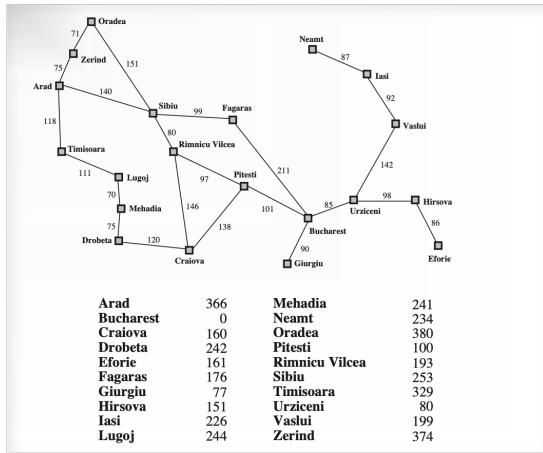
- Expandir nodo más cercano al objetivo, basándose que conduce rápidamente a una solución.
 - Función de evaluación $f(n) = h(n)$.
- Ejemplo h = Distancia en línea recta dos ciudades.

Arad	366	Mehadia	241
Bucharest	0	Neamt	234
Craiova	160	Oradea	380
Drobeta	242	Pitesti	100
Eforie	161	Rimnicu Vilcea	193
Fagaras	176	Sibiu	253
Giurgiu	77	Timisoara	329
Hirsova	151	Urziceni	80
Iasi	226	Vaslui	199
Lugoj	244	Zerind	374

- Algoritmo no óptimo y completo en espacio de estados finitos pero no en espacio de estados infinitos.
- La complejidad espacial y temporal en el peor caso es $O(|V|)$. Siendo $|V|$ el número de vértices.
 - Una buena función heurística puede reducir la complejidad sustancialmente a $O(bm)$. Siendo b el factor de ramificación y m la máxima profundidad del árbol de búsqueda.



Algoritmo voraz (greedy)





Algoritmo voraz (greedy)

(a) The initial state



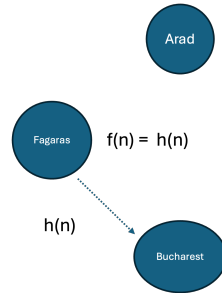
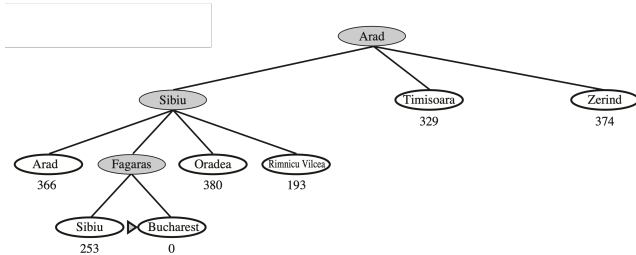
(b) After expanding Arad



Arad	366	Mehadia	241
Bucharest	0	Neamt	234
Craiova	160	Oradea	380
Drobeta	242	Pitesti	100
Eforie	161	Rimnicu Vilcea	193
Fagaras	176	Sibiu	253
Giurgiu	77	Timisoara	329
Hirsova	151	Urziceni	80
Iasi	226	Vaslui	199
Lugoj	244	Zerind	374



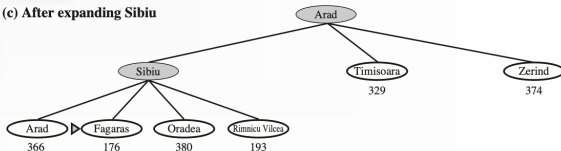
Algoritmo voraz (greedy)





Algoritmo voraz (greedy)

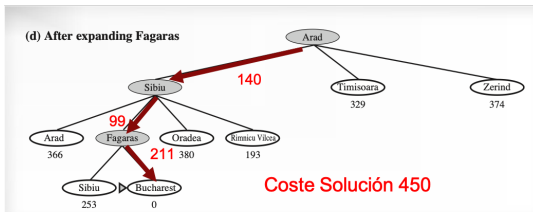
(c) After expanding Sibiu



Arad	366	Mehadia	241
Bucharest	0	Neamt	234
Craiova	160	Oradea	380
Drobeta	242	Pitesti	100
Eforie	161	Rimnicu Vilcea	193
Fagaras	176	Sibiu	253
Giurgiu	77	Timisoara	329
Hirsova	151	Urziceni	80
Iasi	226	Vaslui	199
Lugoj	244	Zerind	374



Algoritmo voraz (greedy)



Arad	366	Mehadia	241
Bucharest	0	Neamt	234
Craiova	160	Oradea	380
Drobeta	242	Pitesti	100
Eforie	161	Rimmicu Vilcea	193
Fagaras	176	Sibiu	253
Giurgiu	77	Timisoara	329
Hirsova	151	Urziceni	80
Iasi	226	Vaslui	199
Lugoj	244	Zerind	374



Ejecución greedy

Nodo seleccionado	Sucesor	$h=f$	Orden en que se han cerrado
Arad	Zerind	374	
-	Timisoara	329	
-	Sibiu	253	(1)
Sibiu	Fagaras	176	(2)
-	Oradea	380	
	Rimmicu	193	
Fagaras	Sibiu	-	cerrado
-	Bucharest	0	(3)
Bucharest	Solución Problema		



Búsqueda A*





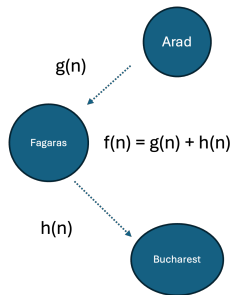
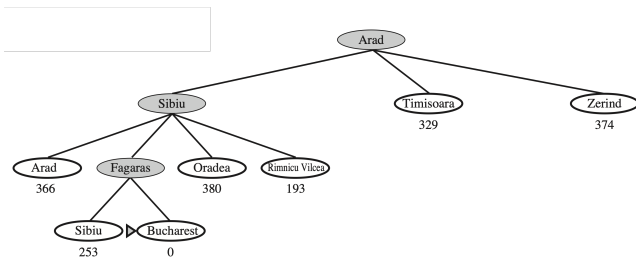
Búsqueda A^*

- Evalúa los nodos combinando $g(n)$, el coste para alcanzar el nodo, y $h(n)$, el coste de ir al nodo objetivo:
 - $f(n)=g(n)+h(n)$
- A^* es óptima si $h(n)$ es una heurística **admisible**, es decir, $h(n)$ nunca sobreestime el coste de alcanzar el objetivo.
 - Diremos que una función heurística h es admisible siempre que se cumpla que su valor en cada nodo sea menor o igual que el valor del coste real del camino que nos falta por recorrer hasta la solución, formalmente: $\forall n \ 0 \leq h(n) \leq h^*(n)$

Arad	366	Mehadia	241
Bucharest	0	Neamt	234
Craiova	160	Oradea	380
Drobeta	242	Pitesti	100
Eforie	161	Rimnicu Vilcea	193
Fagaras	176	Sibiu	253
Giurgiu	77	Timisoara	329
Hirsova	151	Urziceni	80
Iasi	226	Vaslui	199
Lugoj	244	Zerind	374

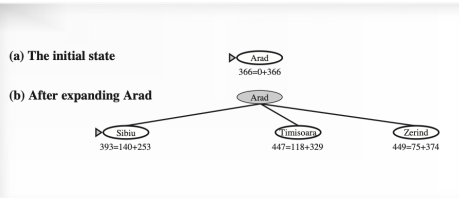


Ejemplo Búsqueda A^*





Ejemplo Búsqueda A^*

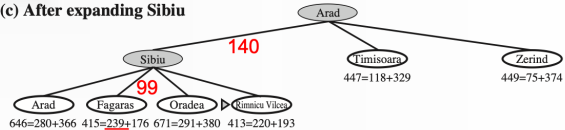


Arad	366	Mehadia	241
Bucharest	0	Neamt	234
Craiova	160	Oradea	380
Drobeta	242	Pitesti	100
Eforie	161	Rimnicu Vilcea	193
Fagaras	176	Sibiu	253
Giurgiu	77	Timisoara	329
Hirsova	151	Urziceni	80
Iasi	226	Vaslui	199
Lugoj	244	Zerind	374

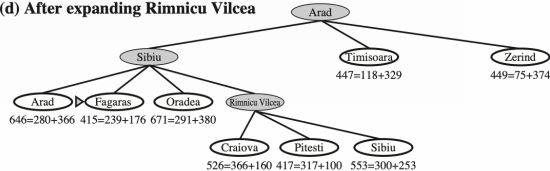


Ejemplo Búsqueda A*

(c) After expanding Sibiu



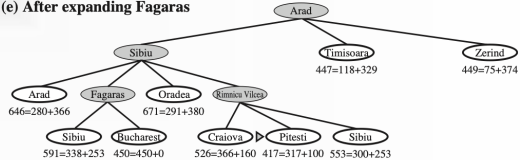
(d) After expanding Rimnicu Vilcea



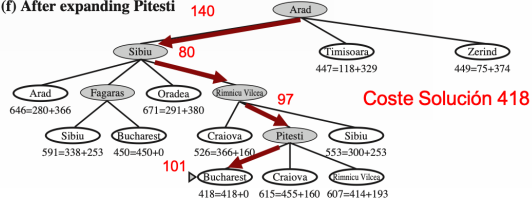


Ejemplo Búsqueda A*

(e) After expanding Fagaras



(f) After expanding Pitesti





Ejecución A*

Nodo seleccionado	Sucesor	$g+h=f$	Orden en que se han cerrado
Arad	Zerind	$75+374= 449$	
-	Timisoara	$118+329= 447$	
-	Sibiu	$140+253=393$	(1)
Sibiu	Fagaras	$(140+99)+176= 415$	(3)
-	Oradea	$(140+151)+380= 671$	
	Rimmicu	$(140+80)+193= 413$	(2)
Rimmicu	Pitesti	$(220+97)+100= 417$	(4)
-	Craiova	$(220+146)+160= 526$	
Fagaras	Sibiu	-	cerrado
-	Bucharest	$(239+211)+0 = 450$	cerrado mejor
Pitesti	Craiova	$(317+138)+160 = 615$	cerrado mejor
-	Bucharest	$(317+101)+0 = 418$	(5)
Bucharest	Solución Problema		



Problemas





Problema 1- Distancias entre ciudades

Las distancias por carretera entre las ciudades de una región vienen dadas en kilómetros en la tabla siguiente:

	A	B	C	D	E	F	G
A		10	15				
B				5	15		
C					5	15	
D							20
E						5	5
F							20
G							

La siguiente tabla contiene una estimación optimista de la distancia en kilómetros desde la ciudad G a cada una de las demás ciudades:

	A	B	C	D	E	F	G
Estimación	19	9	9	4	4	3	0

- a) Aplicar el algoritmo Greedy para encontrar un camino desde la ciudad A a la ciudad G. Indicar para cada iteración el nodo seleccionado para expansión, sus sucesores y los valores correspondientes de $f(n)$. Mostrar el árbol de búsqueda resultante, detallando las operaciones realizadas sobre el mismo.
- b) Aplicar el algoritmo A* para encontrar un camino desde la ciudad A a la ciudad G. Indicar para cada iteración el nodo seleccionado para expansión, sus sucesores y los valores correspondientes de $g(n)$, $h(n)$ y $f(n)$. Mostrar el árbol de búsqueda resultante, detallando las operaciones realizadas sobre el mismo.



Distancias entre ciudades, Greedy

Nodo seleccionado	Sucesor	$h=f$	Orden en que se han cerrado
A	B	9	(1)
-	C	9	
B	D	4	(2)
-	E	4	
D	G	0	(3)
G	SOLUCIÓN	-	-



Distancias entre ciudades, A^*

Nodo seleccionado	Sucesor	$g+h=f$	Orden en que se han cerrado
A	B	$10 + 9 = 19$	(1)
-	C	$15 + 9 = 24$	(3)
B	D	$15 + 4 = 19$	(2)
-	E	$25 + 4 = 29$	cerrado mejor nodo
D	G	$35 + 0 = 35$	cerrado mejor nodo
C	E	$20 + 4 = 24$	(4)
-	F	$30 + 3 = 33$	cerrado mejor nodo
E	F	$25 + 3 = 28$	
-	G	$25 + 0 = 25$	(5)
G	SOLUCIÓN	-	-



Problema 2- Distancias entre ciudades

Aplicar el algoritmo A* para hallar un camino que una las ciudades J y F. Las distancias por carretera entre las ciudades de la región vienen dadas en la tabla siguiente:

	A	B	C	D	E	F	G	H	I	J	K
A			63	75		44					
B			61	77			64				
C				29							
D					38			56			34
E						58					59
F											
G								27		55	
H									35		44
I										40	
J											
K											

y las distancias aéreas a la ciudad F son:

	A	B	C	D	E	F	G	H	I	J	K
F	38	97	59	62	46		111	112	139	158	87

Mostrar el árbol de búsqueda resultante, y detallar en cada paso el nodo seleccionado, así como los valores de $g(n)$, $h(n)$ y $f(n)$ de los nodos generados.



Puzzle 8 - Heurístico

$h(n)$ = número de piezas mal colocadas.

5		8
4	2	1
7	3	6

Estado n

	1	2
3	4	5
6	7	8

Estado final

$$h(n) = 8$$

$h(n)$ = distancia (manhattan) de cada posición a su posición objetivo.

5		8
4	2	1
7	3	6

Estado n

	1	2
3	4	5
6	7	8

Estado final

3		2
1	2	2
1	2	2

$H(n)$

$$h(n) = 15$$



That's all folks!

